23AIE457 Full Stack Development

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

For: StreamVerse — Microservices-Based Video Streaming Platform

Version: 1.0
Date: 5 March 5, 2026

Objective:

To clearly document all functional and non-functional requirements of the StreamVerse video streaming platform, built on a microservices-based MERN stack architecture. This document ensures a shared understanding among stakeholders and serves as a reference for design, development, testing, and maintenance.

Team Information:

Team Member Name	Student Roll No	Project Title
David Shibu	AM.SC.U4CSE23116	StreamVerse — Video Streaming Platform
Abhay Mathummal	AM.SC.U4CSE23102	
Sidharth Vipin Thekkenath	AM.SC.U4CSE23152	

1. Introduction

1.1 Purpose

This document specifies the software requirements for StreamVerse, a microservices-based video streaming platform developed using the MERN stack. The platform enables users to upload, stream, and interact with video content at scale. This SRS is intended for developers, testers, project guides, and evaluators.

1.2 Document Conventions

The following conventions are used in this document:

- FR — Functional Requirement
- NFR — Non-Functional Requirement
- API — Application Programming Interface

- JWT — JSON Web Token
- CDN — Content Delivery Network
- HLS — HTTP Live Streaming
- MQ — Message Queue

1.3 Intended Audience

- Project Guide
- Developers
- Testers
- End Users

1.4 Product Scope

StreamVerse is a full-featured web-based video streaming platform. Users can register, upload videos, stream content with adaptive bitrate, like, comment, subscribe to channels, and receive personalized recommendations. The system is decomposed into independent microservices to ensure high availability, independent scalability, and fault isolation.

1.5 References

- IEEE 830 SRS Standard
- MERN Stack Documentation
- MongoDB Documentation
- React Documentation
- FFmpeg Documentation (Video Transcoding)
- RabbitMQ / Kafka Documentation (Message Broker)
- Socket.io Documentation (Real-time Communication)

2. Overall Description

2.1 Product Perspective

StreamVerse follows a microservices architecture where each core domain is an independent service with its own database and communication contract. Services communicate via REST APIs for synchronous operations and RabbitMQ for asynchronous event-driven tasks.

Layer	Technology
Presentation Layer	React.js + Tailwind CSS
API Gateway	Express.js + NGINX Reverse Proxy
Business Logic Layer	Node.js + Express.js (per service)
Message Broker	RabbitMQ / Apache Kafka
Data Layer	MongoDB (per service database)
Media Storage	AWS S3 / Cloudinary
Video Processing	FFmpeg (transcoding service)

2.2 Product Functions

- User registration, authentication, and profile management
- Video upload, processing, and adaptive bitrate streaming (HLS)
- Channel management and subscription system
- Real-time comments, likes, and notifications
- AI-powered personalized video recommendations
- Search and discovery with filters and trending
- Payment gateway for premium subscriptions
- Admin dashboard for content moderation and analytics

2.3 User Classes and Characteristics

User Role	Access Level	Key Capabilities
Guest User	Public only	Browse and watch public videos, search content
Registered User	Standard access	Upload videos, comment, like, subscribe, manage profile
Premium User	Enhanced access	Ad-free streaming, HD quality, offline downloads
Creator	Creator dashboard	Analytics, monetization, live streaming
Admin	Full system access	Content moderation, user management, system analytics

2.4 Operating Environment

- Client Side — Modern Web Browser (Chrome, Firefox, Safari, Edge)
- Server Side — Node.js runtime (per microservice)
- Database — MongoDB (separate instance per service)
- Message Broker — RabbitMQ
- Container Environment — Docker + Docker Compose
- Reverse Proxy — NGINX

2.5 Design and Implementation Constraints

- Must use MERN stack (MongoDB, Express.js, React.js, Node.js)
- Must implement microservices architecture with service isolation
- Must implement JWT-based authentication
- Must follow REST and event-driven architecture
- Video transcoding must produce at least 360p, 720p, and 1080p variants
- Responsive UI is mandatory (mobile-first design)
- All services must be containerized using Docker

2.6 Assumptions and Dependencies

- Users have stable internet access for video streaming
- MongoDB server instances are available per service
- FFmpeg is available in the transcoding service container

- RabbitMQ broker is running and accessible to all services
- AWS S3 or equivalent object storage is available for media files
- Browser supports HLS streaming (or fallback via hls.js library)

3. System Features (Functional Requirements)

3.1 User Registration & Authentication (FR1)

- The system shall allow new users to create an account using name, email, and password
- The system shall validate email uniqueness across the platform
- The system shall encrypt passwords using bcrypt
- The system shall generate a JWT access token and refresh token upon successful login
- The system shall support Google OAuth 2.0 for social login
- The system shall support password reset via email OTP

3.2 Video Upload & Processing Service (FR2)

- The system shall allow authenticated users to upload video files (MP4, MKV, AVI)
- The system shall queue uploaded videos to the transcoding service via RabbitMQ
- The transcoding service shall convert videos into HLS format at 360p, 720p, and 1080p
- The system shall store processed video segments on cloud object storage (AWS S3)
- The system shall notify the uploader via notification service upon processing completion
- The system shall support video metadata: title, description, tags, category, thumbnail

3.3 Video Streaming Service (FR3)

- The system shall stream videos using HLS adaptive bitrate streaming
- The system shall automatically adjust video quality based on user bandwidth
- The system shall track and persist user watch progress
- The system shall support fullscreen, playback speed, and quality selection controls
- The system shall support embedding of videos via iFrame

3.4 Channel & Subscription Service (FR4)

- Each registered user shall have a personal channel
- Users shall be able to subscribe and unsubscribe from other channels
- The system shall display subscriber count on each channel
- Subscribers shall receive notifications when a new video is uploaded

3.5 Interaction Service — Likes, Comments & Shares (FR5)

- Authenticated users shall be able to like or dislike videos
- Authenticated users shall be able to post, edit, and delete comments
- The system shall support threaded replies to comments
- The system shall allow reporting of inappropriate comments

3.6 Recommendation & Search Service (FR6)

- The system shall provide a personalized video feed based on watch history and subscriptions
- The system shall support full-text search by video title, tags, and description
- The system shall display trending videos based on views and engagement metrics
- Search results shall support filtering by category, upload date, and duration

3.7 Notification Service (FR7)

- The system shall send real-time in-app notifications via Socket.io
- The system shall send email notifications for subscription updates and video processing
- The system shall allow users to configure notification preferences
- Notifications shall be triggered by events published to the message broker

3.8 Payment & Subscription Service (FR8)

- The system shall offer Free and Premium subscription tiers
- The system shall integrate Stripe / Razorpay for secure payment processing
- The system shall handle payment webhooks for subscription activation and renewal
- Premium users shall receive ad-free experience and HD streaming access
- The system shall generate and email payment receipts

3.9 Admin Service (FR9)

- Admin shall be able to view platform-wide analytics (DAU, total streams, revenue)
- Admin shall be able to remove violating content and suspend user accounts
- Admin shall be able to manage content categories and trending tags

4. External Interface Requirements

4.1 User Interface

- Responsive web interface built with React.js and Tailwind CSS
- Video player with HLS.js supporting adaptive bitrate playback
- Real-time notification bell with Socket.io updates
- Creator dashboard with analytics charts (Recharts / Chart.js)
- Drag-and-drop video upload interface with progress indicator

4.2 Hardware Interface

- Standard computer, tablet, or mobile device with a web browser
- Minimum 2Mbps connection for 360p streaming, 10Mbps for 1080p

4.3 Software Interface

Service	Technology	Purpose
API Gateway	Express.js + NGINX	Route and load balance all client requests
Auth Service	Node.js + JWT + bcrypt	Authentication and authorization

Service	Technology	Purpose
Video Service	Node.js + FFmpeg + AWS S3	Upload, transcode, and store videos
Stream Service	Node.js + HLS.js	Adaptive bitrate video delivery
Interaction Service	Node.js + MongoDB	Likes, comments, watch history
Recommendation Service	Node.js + MongoDB	Personalized feed and trending
Notification Service	Node.js + Socket.io + Nodemailer	Real-time and email alerts
Payment Service	Node.js + Stripe	Subscription billing
Message Broker	RabbitMQ	Async event communication between services

4.4 Communication Interface

- HTTP/HTTPS protocol for all REST API communication
- WebSocket (Socket.io) for real-time notification delivery
- AMQP protocol for RabbitMQ message passing between services
- HLS (HTTP Live Streaming) protocol for video delivery

5. Non-Functional Requirements

5.1 Performance Requirements (NFR1)

- System API response time shall be under 300ms for non-media endpoints
- Video playback shall start within 2 seconds on a 10Mbps connection
- The transcoding service shall complete processing of a 10-minute video within 5 minutes
- The system shall support at least 1000 concurrent video streams

5.2 Security Requirements (NFR2)

- All passwords shall be hashed using bcrypt with a minimum salt round of 10
- JWT access tokens shall expire in 15 minutes; refresh tokens in 7 days
- All API routes shall be protected by authentication middleware
- File upload endpoints shall validate MIME types to prevent malicious uploads
- Payment data shall never be stored; Stripe tokenization shall be used
- HTTPS shall be enforced across all services

5.3 Reliability Requirements (NFR3)

- Each microservice shall expose a /health endpoint for monitoring
- The system shall implement circuit breaker pattern (opossum) for inter-service calls
- Failed message queue jobs shall be retried up to 3 times before dead-lettering
- The system shall handle invalid inputs gracefully with descriptive error responses

5.4 Scalability Requirements (NFR4)

- Each microservice shall be independently horizontally scalable via Docker
- The transcoding service shall scale based on upload queue depth
- MongoDB shall use replica sets for data availability
- The system architecture shall support future migration to Kubernetes

5.5 Usability Requirements (NFR5)

- The user interface shall be responsive and functional on screens from 320px to 4K
- Video player controls shall be accessible via keyboard shortcuts
- All forms shall provide real-time inline validation and clear error messages
- Page load time for the homepage shall not exceed 3 seconds

5.6 Maintainability Requirements (NFR6)

- Each service shall follow MVC architecture internally
- All services shall maintain consistent API versioning (/api/v1/...)
- Centralized logging shall be implemented using Winston + Morgan
- Codebase shall follow modular structure with clear separation of concerns

6. Microservices Architecture Summary

Service	Responsibility	Database	Communication
Auth Service	Registration, login, JWT, OAuth	MongoDB: users	REST
Video Service	Upload, metadata, thumbnails	MongoDB: videos	REST + MQ Publisher
Transcoding Service	FFmpeg processing, HLS generation	MongoDB: jobs	MQ Consumer
Stream Service	Video delivery, watch progress	MongoDB: progress	REST
Channel Service	Channel CRUD, subscriptions	MongoDB: channels	REST
Interaction Service	Likes, comments, replies	MongoDB: interactions	REST
Recommendation Service	Feed, trending, search	MongoDB: analytics	REST
Notification Service	In-app, email, push alerts	MongoDB: notifications	MQ Consumer + WS
Payment Service	Stripe billing, webhooks	MongoDB: payments	REST + Webhook
API Gateway	Routing, rate limiting, auth check	N/A	REST (all services)